

TalentSignal Revenue Engine

Revision & Q&A study guide · Releases 0–3 · everyday-analogy edition

How to use this. Read it once. Then cover the answers and say each idea out loud in your own words. If you get stuck, use the **green picture** — every idea has one from everyday life. You don't need to memorize the tech; you need to be able to explain it like you'd explain it to a friend.

Part 1 — the whole project in one breath

What it is. TalentSignal helps a staffing agency — a company that finds workers for other companies — notice which companies are about to need staff *before* they announce it. So the salesperson can call first, before rival agencies even know there's a job.

*Picture: it's a **dating app, but for jobs** — it matches companies that need people with people who need work.*

The one rule that runs everything. The app can write and suggest, but it can never send anything out on its own. A human always reviews and clicks 'send.'

*Picture: like your **phone suggesting a text message**. Your phone writes the words, but nothing is sent until you press send. The app works exactly the same way — it writes, you decide to send.*

Part 2 — what each release does

R0 — Walking skeleton (first 3 stories)

Getting the basic machine running: the screen you look at, the 'brain' that does the thinking, and the memory that stores everything — all connected and talking. Plus a secure login. It looks boring on purpose.

Picture: Like building a house: you put up the frame and the pipes first, and decorate the rooms much later. Nobody claps at a frame, but the whole house stands on it.

R1 — The smart part (next 4 stories)

Now it gets clever. It ranks which companies probably need staff, ranks which people best fit a job, and gives each a score — and it always shows the reasons behind every number.

Picture: Like guessing if it will rain: dark clouds, strong wind, and it rained yesterday — the more signs, the more sure you are. The app adds up the clues the same way.

R2 — The heart: a human is always in charge (next 4 stories)

The app writes a ready-to-send outreach message — but saves it as a draft and sends it nowhere. Only when a person reviews it and clicks 'release' does it count, and the app writes down who did it and when.

Picture: Like writing an essay: you can draft and edit all you want, but it only counts once you click 'submit.' The app drafts; you submit.

R3 — The manager's dashboard (started)

A page of numbers a boss cares about: how many placements each month, how long hiring takes, and an overall 'demand' number — shown as totals and averages only, never any one person's private details.

*Picture: Like a report card that shows the **class average** but never any single student's grade. You see the big picture, not the individuals.*

Part 3 — the ideas you should be able to explain

Confidence score

A number from 0 to 1 that says how sure the app is that a company secretly needs staff. It adds up little clues — a job posted again, a role open a long time, no salary listed — and shows them, so it's never just a mystery number.

Picture: Like guessing the weather — you add up the signs (clouds + wind + yesterday's rain) to decide how sure you are.

The audit log (the permanent record)

Every important action gets written down forever: who did it, when, and what changed. Nobody — not even a boss — can go back and edit or delete those records. The database itself refuses.

*Picture: Like a **WhatsApp group chat**: you can send new messages, but you can't delete or edit the old ones. The history stays exactly as it happened.*

The human-in-the-loop 'send' gate

The app writes the outreach and stores it as a draft, delivered nowhere. Only a person's click marks it as released, saving who and when. Nothing is actually mailed out — the 'released' label is the line the app will never cross by itself.

Picture: Like your phone suggesting a text — it writes it, but only you press send.

Aggregates only (no private details)

The dashboard shows counts and averages — never one person's name or phone number. Private details never reach the reports page.

Picture: Like a report card showing the class average, but never naming a single student's grade.

Matching people to jobs

It ranks candidates for a job by how much their skills overlap what the job needs, and shows which skills matched. More overlap, higher match.

Picture: Like a dating app matching two people by shared interests — the more you have in common, the higher the match.

'Fake data behind a clean wall'

The outside job listings are pretend for now, but they sit behind a clean boundary. When a real data feed is ready, it plugs in by changing one line — nothing else changes.

*Picture: Like learning to drive in an **empty parking lot**: the car, the steering, the brakes are all real — only the traffic is missing. When you're ready you drive on the real road, and the car doesn't change at all.*

Part 4 — questions you might be asked

Cover the answers. Try each one in your own words first.

Q — Did you build this, or did the AI?

The AI wrote most of the code. My job was to say what to build, check it, and decide if it was right — and I turned plenty of it down. More than once it told me something was finished when it wasn't, and catching that is the actual job.

Q — Why does it look so plain?

On purpose. I made the important parts work correctly first — the safety and the trust rules — before making it pretty. The dashboard is the first real visual screen.

Q — How do you know the records can't be faked or changed?

The database itself blocks any attempt to edit or delete a record — even a boss can't. I proved it with a test that tries to change and delete a record and shows the database refuse both times.

Q — Where did the scoring numbers come from?

From me. They're my suggested values, clearly marked as waiting for the client's approval — not final. The instructions said these are a business decision I shouldn't just invent quietly, so I proposed sensible numbers openly and kept them in one place that's a one-line change if the client wants different ones.

Q — Is it secure?

Logins yes — passwords are scrambled and never stored plainly, sessions expire, and the login gives nothing away about who has an account. Full permission rules aren't finished yet — that's the next release — and nothing is online yet, so it hasn't faced the real internet.

Q — Can it handle a thousand people at once?

I haven't tested that, so I won't claim a number. Speed-testing is a separate later task. I'd rather tell you that honestly than guess.

Q — What's real and what's fake?

The app is real — real database, real login, real screens. Only the outside job listings are pretend for now, behind a clean wall, so a real feed plugs in with one line.

Q — Why isn't the AI allowed to send things itself?

That's the main rule: the AI helps, a human decides. Anything that would leave the app needs a person to approve it — and that's built into the code, not just a promise. It protects the agency's relationships and people's trust.

Q — What was the hardest problem you caught?

A safety limit that was accidentally applied to the whole app instead of just the login — a couple of page refreshes would have frozen everything with errors, right in a demo. It passed every test because the tests only checked the part that was supposed to work, not the side effect. I only found it by clicking through the real app myself.

Q — What would you do differently?

Set up my tools on day one — a missing install cost me time early. And I'd have asked the client about going live sooner, since that's the one part I can't do alone.

Three things to remember walking in.

- Start with the trust story, not the tech — 'the app writes, a human sends' is the whole pitch.
- Say what you *haven't* proven before anyone asks — it makes everything else believable.
- If you blank on an idea, reach for its green picture — that's what it's there for.